ECE 410/510 — Hardware for AI/ML

Codefest cf07

Pratibha Munnangi

(a) Dense & Sparse Compute / Memory Expressions

Quantity	Expression	Value at N = 512
Dense compute	FLOPs = $2N^2$ (1 mul + 1 add / element)	$2 \cdot 512^2$ = 524,288 FLOPs
Dense memory	Bytes = $4N^2$ (FP32, 4 B / element)	$4 \cdot 512^2$ = 1,048,576 B ≈ 1 MiB
Sparse compute	FLOPs(s) = $2(1 - s)N^2$	524288(1-s)FLOPs
Sparse memory	Bytes(s) = $8(1 - s)N^2 + 4(N + 1)$	function of s

CSR sparse-memory breakdown:

- values array — 4 B per non-zero → $4(1 - s)N^2$
- column-index array — 4 B (INT32) per non-zero → $4(1 - s)N^2$
- row-pointer array — 4 B (INT32) × (N + 1) → $4(N + 1)$
- Total: $8(1 - s)N^2 + 4(N + 1)$

(b) FLOPs Speedup & Sparsity for 2×

The FLOPs speedup is the ratio of dense to sparse compute:

$$\text{Speedup\_FLOPs} = 2N^2 / [\,2(1-s)N^2\,] = 1/(1-s)$$

The N^2 cancels — compute speedup depends only on sparsity, not matrix size. Setting it equal to 2×:

$$1/(1-s) = 2 \;\Rightarrow\; 1 - s = 0.5 \;\Rightarrow\; s = 0.50$$

Result: 50% sparsity yields a 2× compute speedup.

(c) Memory Breakeven Sparsity — Derivation

Breakeven is the sparsity at which CSR storage equals dense storage:

$$8(1-s)N^2 + 4(N+1) = 4N^2$$

Solve for s:

$$8(1-s)N^2 = 4N^2 - 4(N+1)$$

$$1 - s = [\,4N^2 - 4(N+1)\,] / 8N^2 = (N^2 - N - 1) / 2N^2$$

$$\text{s\_break} = 1 - (N^2 - N - 1)/2N^2 = (N^2 + N + 1) / 2N^2$$

Substituting N = 512:

$$\text{s\_break} = (512^2 + 512 + 1) / (2 \cdot 512^2) = 262{,}657 / 524{,}288 \approx 0.5010$$

Result: memory breakeven ≈ 50.1% sparsity. CSR stores 8 B per surviving element (4 B value + 4 B column index) versus 4 B dense, so at least half the elements must be zero just to break even; the small row-pointer term 4(N+1) pushes the exact point slightly above 50%. Above ~50.1% sparsity, CSR uses less memory than dense.

(d) End-to-End Speedup at s = 0.9 (Memory-Bound)

System: memory-bandwidth-limited, BW = 320 GB/s, N = 512. In this regime execution time is set by bytes moved: t = Bytes / BW.

Dense

$$\text{t\_dense} = 4N^2 / \text{BW} = 1{,}048{,}576 / (320\times10^9) \approx 3.277\ \mu\text{s}$$

Sparse at s = 0.9

$$\text{Bytes\_sparse} = 8(0.1)(512^2) + 4(513) = 209{,}715 + 2{,}052 = 211{,}767\ \text{B}$$

$$\text{t\_sparse} = 211{,}767 / (320\times10^9) \approx 0.662\ \mu\text{s}$$

End-to-end speedup

$$\textbf{Speedup} = \textbf{t\_dense} / \textbf{t\_sparse} = \textbf{1,048,576} / \textbf{211,767} \approx \textbf{4.95}\times$$

Interpretation: the compute speedup at s = 0.9 would be 1/(1−s) = 10×, but the memory-bound end-to-end speedup is only ≈4.95×. Each surviving weight carries a 4 B column index alongside its 4 B value, doubling per-element memory cost. Memory volume therefore drops by ~5× rather than 10×, and in a bandwidth-limited system that memory factor — not the FLOPs factor — caps the achievable wall-clock speedup.

Summary

Result	Value
Dense compute	$2N^2$ = 524,288 FLOPs
Dense memory	$4N^2$ = 1,048,576 B
Sparse compute	$2(1 - s)N^2$
Sparse memory	$8(1 - s)N^2 + 4(N + 1)$
FLOPs speedup	$1 / (1 - s)$
Sparsity for 2× compute	s = 0.50
Memory breakeven	$s = (N^2 + N + 1)/2N^2 \approx 0.5010$
End-to-end speedup, s = 0.9, memory-bound	≈ 4.95×